

Day 1 Live-Coding Session — 5 Problems

Five String module problems for the combined-audience Day 1 session. Titles are given as their in-story scenario name rather than the generic textbook name, so they read fresh to students. Please solve each end-to-end and confirm sample outputs before presenting.

1. The College Coding Arcade

Core problem: Rock-Paper-Scissors Game

Scenario

The coding club is setting up a mini arcade module for orientation day, where new students play a quick Rock-Paper-Scissors match against the computer as a warm-up before their placement round. The club needs a simulator that plays several rounds between the "player" and the "computer," records the outcome of every round, and prints a final scoreboard summarizing wins, losses, draws, and the player's win percentage.

Task

- Generate the computer's move randomly (Rock, Paper, or Scissors) for each round.
- Accept the player's move for each round (via input, or a predefined list for a live demo).
- Determine and display the winner of each round using standard Rock-Paper-Scissors rules.
- Repeat for N rounds (suggested N = 5).
- After all rounds, print a summary table: Round | Player Move | Computer Move | Result.
- Print the total Wins, Losses, Draws, and the player's Win Percentage.

Suggested method signature(s): String playRound(String playerMove, String computerMove)

Sample Input / Output

Input	Output
Round 1 — Player: Rock, Computer: Scissors	Player Wins
Round 2 — Player: Paper, Computer: Paper	Draw
Round 3 — Player: Scissors, Computer: Rock	Computer Wins
Final Summary (after 5 rounds)	Wins: 2 Losses: 2 Draws: 1 Win % = 40.0%

Concepts covered: Random number generation, conditional logic, loops, arrays for the round table, formatted/tabular output, basic statistics (percentage calculation).

2. The QA Text Verification Toolkit

Core problem: *Palindrome Checker (3 Approaches)*

Scenario

The QA team on a coding-assessment platform is building a small internal toolkit to sanity-check palindrome-detection logic before it ships to students as a graded exercise. To make sure the logic is solid regardless of implementation style, the toolkit must verify a given text using three independent approaches — iterative comparison, recursion, and array reversal — and confirm all three agree on the result.

Task

- Accept a text input (e.g., a word or short phrase).
- Implement an iterative check: compare characters from both ends moving toward the middle.
- Implement a recursive check: recursively compare the first and last characters, shrinking the substring each call.
- Implement an array-reversal check: convert the string to a character array, reverse it, and compare it to the original.
- Print the result of all three approaches for the same input — they should always agree.

Suggested method signature(s): `boolean isPalindromIterative(String text) + boolean isPalindromeRecursive(String text) + boolean isPalindromeArrayReversal(String text)`

Sample Input / Output

Input	Output
"madam"	<i>Iterative: Palindrome Recursive: Palindrome Array Reversal: Palindrome</i>
"hello"	<i>Iterative: Not Palindrome Recursive: Not Palindrome Array Reversal: Not Palindrome</i>

Concepts covered: Loops, recursion, array manipulation, string comparison, comparing implementation trade-offs.

3. The VIP Customer Identification System

Core problem: VIP Customer / Prefix Check

Scenario

You are developing a CRM application that identifies VIP customers based on a predefined naming convention. All VIP customer IDs begin with the prefix "VIP-". Before displaying exclusive offers, the system must verify whether the entered customer ID follows this standard. IDs that do not begin with the required prefix should be treated as regular customer accounts.

Task

- Develop a method that validates customer IDs.
- The method takes one input argument: `customerId`.
- It should verify whether the ID starts with "VIP-" and return "VIP Customer" if it satisfies the rule; otherwise, return "Regular Customer".
- Call this method from main and print the result.

Suggested method signature(s): `String validateCustomerId(String customerId)`

Sample Input / Output

Input	Output
"VIP-4589"	VIP Customer
"CUS-4589"	Regular Customer

Concepts covered: `startsWith()`, string comparison, conditional logic.

4. The Unique Letter Hunt Mini-Game

Core problem: First Non-Repeating Character

Scenario

A code-learning platform runs a light-hearted "Unique Letter Hunt" mini-game between exercises: a player types any word or sentence, and the app must instantly find and highlight the first character that appears only once in the entire input — a fun, quick way to reinforce character-frequency thinking between coding drills.

Task

- Accept an input string (a word or short sentence).
- Compute the frequency of every character in the string.
- Scan the string left to right and find the first character whose frequency is exactly 1.
- Print that character, or a clear message if no non-repeating character exists.

Suggested method signature(s): `char findFirstNonRepeatingChar(String text)`

Sample Input / Output

Input	Output
"swiss"	<i>First Non-Repeating Character: 'w'</i>
"aabbcc"	<i>No Non-Repeating Character Found</i>

Concepts covered: Character frequency counting, loops, HashMap/array-based counting, early-exit scanning.

5. The Newsroom Copy-Editing Desk

Core problem: Shortest and Longest Word Finder

Scenario

A newsroom's digital copy-editing tool helps editors scan a draft article in seconds and flag the shortest and longest words in it — catching stray typing fragments or half-finished words, and spotting unusually long jargon that might not fit a tight headline — before the piece goes to print.

Task

- Accept a sentence or short paragraph as input.
- Split the text into individual words (handle spacing/punctuation sensibly).
- Scan through the words to find the shortest and the longest one.
- Print both words along with their lengths.

Suggested method signature(s): `String[] findShortestAndLongestWord(String text)`

Sample Input / Output

Input	Output
<i>"Java is an object oriented programming language"</i>	<i>Shortest: "is" (2) Longest: "programming" (11)</i>

Concepts covered: String splitting (`split()`), loops, comparison, tracking min/max values.